

Lab 05 - Stream Ciphers in Software

In this lab, you will explore the ChaCha20 stream cipher. It is a modern encryption algorithm designed for speed, simplicity, and security. Stream ciphers like ChaCha20 are used to transform plaintext data into ciphertext using a secret key and a unique nonce, ensuring confidentiality. Unlike block ciphers, which encrypt fixed-size blocks, stream ciphers generate a keystream to encrypt data of any size, making them ideal for encrypting files of variable length, including text, images, or binary files.

It was created by Daniel J. Bernstein as a fast and secure alternative to other encryption algorithms like AES, especially for use in software implementations that don't have hardware acceleration for AES. It is efficient, resistant to known attacks, and offers excellent performance on a wide variety of devices, including mobile platforms.

ChaCha20 is part of the ChaCha20-Poly1305 authenticated encryption standard and is widely used in modern cryptographic systems, including secure communications (e.g., VPNs, TLS). Poly1305 provides the integrity and authentication checks which ChaCha20 by itself is missing, so you don't find ChaCha20 commonly used as a standalone encryption algorithm. It is used by itself as a pseudorandom number generator. For the purposes of this lab, you will use it as a standalone encryption algorithm.

There are three parts to this lab. You will encrypt, decrypt, and recover plaintext using an attack against ciphertext where the nonce and the key were reused.

Part 01 - Encrypt

1. Copy the skeleton code from `repo.homework.3310.com` using the `repo/repo` credentials. You need to adjust the permissions so you can write in the directories. This will contain the skeleton code, plaintext, and ciphertext for all three parts of Lab 05.

```
scp -r repo@repo.homework.3310.com:/home/repo/lab05 .
```

2. While there are multiple libraries available that implement cryptographic primitives for python, you will be using `Cryptography` to work with ChaCha20. It is a popular library because it offers high-level functions, as well as the low-level primitives. You may need to install it on your Ubuntu desktop machine.

```
pip install cryptography
```

OR

```
sudo apt install python3-cryptography
```

3. Modify the `part01_02_encrypt_decrypt_chacha_skel.py` code so it can

encrypt the plaintext found in `part01_plaintext.txt`. The skeleton of the encrypt and decrypt functions has been annotated so you can understand what should be used from the cryptography library for python. Look for the TODO: notifications. You will find <https://cryptography.io/en/3.0/hazmat/primitives/symmetric-encryption/> useful. You will also need to modify the `loadKeyNonce` function and then correctly use the functions to encrypt (in `part01`) and decrypt (in `part02`).

Example plaintext in `part01_plaintextDemo.txt` is below. It is found in the Demo directory. For your lab, **you will use** the plaintext found in `part01_plaintext.txt` Not the following. This is an **EXAMPLE ONLY**.

Why do Ewoks always ask you to pay their tab? They're always a little short.

4. As shown below, the syntax for encryption is

```
python3 part01_02_encrypt_decrypt_chacha_skel.py encrypt
<inputFile> <outputFile> <keyFile> <nonceFile>
```

Specifically for the example:

```
python3 part01_02_encrypt_decrypt_chacha_skel.py encrypt
part01_plaintextDemo.txt part01_ciphertextDemo.bin
part01_keyDemo.bin part01_nonceDemo.bin
```

5. You may want to test your work by decrypting your output. This is not a mandatory step, but it may ensure you have everything working properly. Be sure not to overwrite your original `plaintext.txt` file with your testing.

Specifically for the example:

```
python3 part01_02_encrypt_decrypt_chacha_skel.py decrypt
part01_ciphertextDemo.bin part01_decrypted_plaintextDemo.txt
part01_keyDemo.bin part01_nonceDemo.bin
```

6. **Print the `part01_ciphertext.bin` file and include a screenshot of the result in your lab report.** Use `xxd -b <outputFile>` to display both the binary and ASCII. `part01_ciphertextDemo.bin` is below. Your output from the lab will be different because you have different plaintext, nonce, and key for the actual `part01` lab exercise.

```
cp3310@cp3310:~/homework/lab05/part01$ xxd -b Demo/part01_ciphertextDemo.bin
00000000: 01000000 00010001 11001110 01011111 00001001 01011110 @...^
00000006: 00110000 00000101 11111101 11100110 10010000 11000110 0....
0000000c: 00110100 11110011 11010000 11111011 11001111 00110011 4...3
00000012: 00101011 10110111 11101101 10111000 00111101 01001011 +...=K
00000018: 11011111 00001010 00010111 00001101 10001111 01011000 ...X
0000001e: 11000011 01011111 01101111 00010001 11111011 10100011 ._o...
00000024: 01111110 11110010 01000010 00101001 10101100 00011110 ~.B)...
0000002a: 01110100 01011101 01011111 11111001 10100011 11111000 t]_...
00000030: 10001011 00111010 11101100 00000110 00011010 01101100 :...l
00000036: 10100000 00101010 10011010 01010101 00000101 10010100 *.U..
0000003c: 10100010 01000111 10000111 11110100 01001101 10101010 .G..M.
00000042: 10101110 00100001 00000101 11001011 01000101 00000111 !..E.
00000048: 11101100 11110110 10000010 00010110 ....
```

Part 02 - Decrypt

7. Use the `part01_02_encrypt_decrypt_chacha_skel.py` code to decrypt the ciphertext found in `part02_ciphertext.bin`. The plaintext should be displayed on your screen. **Include a screenshot of the resulting printout on your screen in your lab report.** This should be human readable.

Remember, you will be using a different ciphertext.bin, nonce, and key in your actual lab assignment.

```
python3 part01_02_encrypt_decrypt_chacha_skel.py decrypt  
<inputFile> <outputFile> <keyFile> <nonceFile>
```

Specifically for the example:

```
python3 part01_02_encrypt_decrypt_chacha_skel.py decrypt  
part02_ciphertextDemo.bin part02_plaintextDemo.txt  
part02_keyDemo.bin part02_nonceDemo.bin
```

```
File 'part02_example_cipherText.bin' decrypted successfully to 'part02_example_plainText_out.txt'.  
Decrypted content (ASCII): How do Ewoks communicate over long distances?  
With Ewokie talkies.
```

8. **Upload your working <netid>_part01_02_encrypt_decrypt_chacha_skel.py code as a separate file.**

Part 03 - Key and Nonce Reuse

In this part of the lab, you will explore a critical vulnerability in stream ciphers like ChaCha20: the reuse of a nonce. A nonce (number used once) is essential for ensuring that each encryption operation produces a unique ciphertext, even when the same key is used. When the same nonce is reused with the same key to encrypt different plaintexts, it creates a serious security flaw that can allow an attacker to recover the original plaintexts.

If the same nonce is used to encrypt multiple plaintexts, the keystream remains the same, leading to a situation where XORing the two ciphertexts will cancel out the keystream, leaving behind the XOR of the two plaintexts. Under a known plaintext attack where plaintext1 is known, it then becomes trivial to recover plaintext2.

9. You have been given two ciphertext files (`part03_ciphertext1.bin` and `part03_ciphertext2.bin`) and one of the original plaintext files (`part03_plaintext1.txt`). With this information, you will recover the second plaintext file.
10. Modify the skeleton code `part03_xor_nonce_reuse_skel.py`. Most of the heavy lifting has been done for you through the implementation of the `xorBytes` function. It has been annotated to help you understand what it is doing.

Look for the TODO: notifications. You will need to read the filenames from the command line, read the files in as binary data using the `readFile` function, XOR the two ciphertexts using the `xorBytes` function, then XOR that result with the known plaintext (`part03_plaintext1.txt`). That will yield the recovered plaintext (`part03_plaintext2.txt`)

11. Once you have your code modified, run the program

```
python3 part03_xor_nonce_reuse_skel.py <cipherTextFile1>  
<cipherTextFile2> <knownPlainTextFile> <recoveredPlainTextFile>
```

Specifically, for the lab (there is no Demo for this part) the syntax is

```
python3 part03_xor_nonce_reuse_skel.py part03_ciphertext1.bin  
part03_ciphertext2.bin part03_plaintext1.txt  
part03_recovered_plaintext2.txt
```

12. The `part03_plaintext2.txt` file should be displayed on your screen. **Include a screenshot of the resulting printout on your screen in your lab report.** This should be human-readable.
13. **Would a unique keystream be generated if the nonce was reused, but the key changed? Would you really want to implement ChaCha20 reusing the nonce? Why or why not?**
14. **Upload your working <netid> part03_xor_nonce_reuse_skel.py code as a separate file.**

Lab 05 Template

Part 01

- 1) **Print the `part01_ciphertext.bin` file and include a screenshot of the result in your lab report.**
(15 points)

Part 02

- 2) Include a screenshot in your lab report of the resulting ASCII text printed on your screen when you decrypted from part02_ciphertext.bin.
(15 points)
- 3) Upload your working <netid>_part01_02_encrypt_decrypt_chacha_skel.py code as a separate file.
(20 points)

Part 03

- 4) Include a screenshot in your lab report of the resulting ASCII text printed on your screen, which is part03_plaintext2.txt printed on your screen.
(15 points)
- 5) Questions related to reusing nonce, but not the key
 - a. Would a unique keystream be generated if the nonce was reused, but the key changed?
(5 points)
 - b. Would you really want to implement ChaCha20 reusing the nonce? Why or why not?
(10 points)
- 6) Upload your working <netid>_part03_xor_nonce_reuse_skel.py code as a separate file.
(20 points)